

1. DBMS vs File System

1.1 File System

A file system stores data in separate files managed by the operating system.

Example:

Student.txt

Employee.txt

Marks.txt

Fees.txt

Programs directly access and manipulate these files.

Problems with the File System

1. Data Redundancy

- Same data may be stored in multiple files.
- Wastes storage.
- Can cause inconsistency.

Example:

Student.txt → Harshali, Pune

Fees.txt → Harshali, Mumbai

The same student's information is inconsistent.

2. Data Inconsistency

- Occurs when multiple copies of the same data contain different values.
- Usually caused by redundancy.

3. Difficulty in Data Access

- Complex queries require writing separate programs.
- No general-purpose query language like SQL.

4. Data Isolation

- Data may be distributed across different files and formats.
- Combining information becomes difficult.

5. Integrity Problems

- Difficult to enforce rules consistently.

Example:

Age must be greater than 0

Student_ID must be unique

6. Atomicity Problems

If a transaction partially executes and fails, recovering the database consistently is difficult.

7. Concurrent Access Problems

Multiple users accessing/updating the same file can produce inconsistent results.

8. Security Problems

Fine-grained access control is difficult.

9. Backup and Recovery Problems

Failure recovery is less sophisticated.

10. Program-Data Dependence

Programs are often tightly coupled to file structures.

1.2 DBMS

A Database Management System is software that allows users/applications to:

- define data

- create databases
- store data
- retrieve data
- update data
- delete data
- enforce constraints
- control access
- manage transactions
- recover from failures

Examples:

- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- SQLite

1.3 Advantages of DBMS

Reduced Data Redundancy

DBMS can reduce unnecessary duplication.

Improved Data Consistency

Centralized management helps maintain consistent data.

Data Sharing

Multiple users/applications can access the database.

Security

Authorization and privileges can control access.

Integrity

Constraints enforce valid data.

Concurrency Control

Multiple transactions can execute safely.

Backup and Recovery

DBMS provides mechanisms for failure recovery.

Data Independence

Changes at one level can often be isolated from higher levels.

1.4 DBMS vs File System — Exam Table

Feature	File System	DBMS
Data redundancy	Generally higher	Can be controlled
Data consistency	Difficult	Better support
Security	Limited	Stronger mechanisms
Concurrency	Limited	Supported
Recovery	Limited	Supported
Integrity constraints	Limited	Supported
Complex queries	Difficult	SQL/query languages
Data independence	Low	Supported
Data sharing	Difficult	Easy
Transaction management	Limited	Supported

Exam Trap

✗ "DBMS eliminates all redundancy."

Correct:

- ✓ DBMS **controls/reduces** redundancy.
-

2. Database Architecture

Database architecture describes how users/applications interact with the database system.

Three common deployment architectures are:

1. One-tier
 2. Two-tier
 3. Three-tier
-

2.1 One-Tier Architecture

Application and database are on the same system.

User



Application



Database

Example:

A local application using SQLite.

Characteristics

- Simple
 - Suitable for standalone applications
 - Limited scalability
 - No separate database server required
-

2.2 Two-Tier Architecture

The client directly communicates with the database server.

Client



Database Server

The client generally contains the user interface and application logic.

Example

Desktop Application



MySQL

Advantages

- Simple client-server model
- Easy to implement for small systems

Disadvantages

- Business logic may reside heavily on clients.
 - Scaling many clients can be difficult.
 - Greater exposure of database server.
-

2.3 Three-Tier Architecture

There are three layers:

Client



Application Server



Database Server

Example:

Browser



Django Application



MySQL

Layer 1 — Presentation Layer

Responsible for:

- user interface
- displaying information
- receiving user input

Example:

HTML/CSS/JavaScript.

Layer 2 — Application/Business Logic Layer

Responsible for:

- business rules
- validation
- authentication
- processing requests

Example:

Django/Java/Spring/.NET application.

Layer 3 — Database Layer

Responsible for:

- storing data

- retrieving data
- updating data

Example:

MySQL/PostgreSQL.

Advantages

- Better security
- Better scalability
- Easier maintenance
- Separation of concerns
- Business logic is centralized

Exam Trap

Three-tier architecture ≠ Three-schema architecture.

They are different concepts.

3. Three-Schema Architecture

Three-schema architecture describes **different levels of database representation**.

It was designed to provide:

- data abstraction
- data independence
- separation between user views and physical storage

Three levels:

External Level



Conceptual Level



Internal Level

3.1 External Level

Also called:

View level

It describes how individual users or applications see the database.

Different users can have different views.

Example:

Teacher

Student_ID

Name

Marks

Accounts Department

Student_ID

Name

Fees

Both are using the same database but see different views.

Purpose

- simplicity
 - security
 - customized views
-

3.2 Conceptual Level

Also called:

Logical level

Describes the complete logical structure of the database.

Example:

STUDENT(Student_ID, Name, Age, Dept_ID)

DEPARTMENT(Dept_ID, Dept_Name)

COURSE(Course_ID, Course_Name)

It describes:

- entities
- attributes
- relationships
- constraints

It does not describe detailed physical storage.

3.3 Internal Level

Also called:

Physical level

Describes how data is physically stored.

Includes:

- storage structures
- file organization
- indexes
- disk blocks

- record placement
-

3.4 Easy Memory Trick

External → What users see

Conceptual → What database logically contains

Internal → How data is physically stored

4. Data Abstraction

Data abstraction means:

Hiding unnecessary implementation details from users.

The user does not need to know how the database physically stores information.

Levels of Abstraction

Physical Level

Describes:

How data is stored.

Example:

- disk blocks
- indexes
- file organization

Logical Level

Describes:

What data is stored and how the data is related.

Example:

Student

Course

Department

View Level

Describes:

What a particular user sees.

Why Data Abstraction?

It provides:

- simplicity
 - security
 - easier database usage
 - separation between implementation and user interface
-

5. Data Independence

Data independence means:

The ability to change the schema at one level without requiring changes at the next higher level.

There are two types:

1. Physical Data Independence
 2. Logical Data Independence
-

5.1 Physical Data Independence

Ability to change the **internal/physical schema** without changing the conceptual schema.

Example:

Change:

File organization

from:

Sequential

to:

Indexed

The logical structure remains the same.

Internal changes



Conceptual unchanged

Keywords

If a question mentions:

- storage
- disk
- indexes
- file organization
- physical structure

Think:

Physical Data Independence

5.2 Logical Data Independence

Ability to change the **conceptual schema** without changing external views.

Example:

Original:

STUDENT(ID, Name, Age)

Later:

STUDENT(ID, Name, Age, Email)

Existing views can continue working without modification.

Conceptual changes



External views unchanged

Keywords

If a question mentions:

- tables
- attributes
- relationships
- logical schema

Think:

Logical Data Independence

5.3 Important Exam Fact

Generally:

Logical data independence is harder to achieve than physical data independence.

Why?

Because changes to the conceptual schema can affect multiple external views and applications.

6. Database Models

A database model defines:

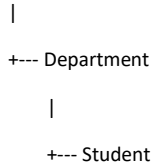
How data, relationships and constraints are represented.

Major database models:

6.1 Hierarchical Model

Data is represented as a tree.

University



Characteristics:

- Parent-child structure
- One parent can have many children
- Suitable for hierarchical data

Limitation

Many-to-many relationships are difficult to represent naturally.

6.2 Network Model

Data is represented using a graph/network structure.

Characteristics:

- More flexible than hierarchical model
- A record can have multiple relationships
- Supports complex relationships

6.3 Relational Model

Data is represented using tables.

Example:

Student_ID	Name	Department
------------	------	------------

101	A	CS
102	B	IT

Important concepts:

- relation
- tuple
- attribute
- domain
- keys
- constraints

This is the **most important model for your current DBMS preparation.**

6.4 Object-Oriented Model

Data is represented as objects.

Supports concepts such as:

- classes
- objects
- inheritance
- encapsulation

Useful for applications heavily based on object-oriented programming.

7. Relational Model

The relational model represents data as **relations**.

A relation is commonly represented as a table.

Example:

STUDENT

Student_ID Name Age Dept_ID

101 A 22 10

102 B 23 20

103 C 21 10

Important properties of a relation

In the classical relational model:

- Each cell contains a single atomic value.
- Values in an attribute come from an appropriate domain.
- Tuples are logically distinct.
- Attribute names identify columns.
- The order of rows is not semantically important.
- The order of columns is not semantically important.

Important distinction

A database table in SQL may permit things that the pure relational model does not emphasize in exactly the same way, such as duplicate rows depending on context.

For exam purposes, remember the **classical relational model** assumes tuples are distinct.

8. Relation, Tuple, Attribute and Domain

Consider:

Student_ID Name Age Department

101 A 22 CS

Student_ID Name Age Department

102 B 23 IT

Relation

The entire table:

STUDENT

is a relation.

Tuple

A row is a tuple.

Example:

(101, A, 22, CS)

Attribute

A column is an attribute.

Examples:

Student_ID

Name

Age

Department

Domain

A domain is the set of permitted/appropriate values for an attribute.

Example:

Age → positive integers

Department → CS, IT, Mechanical, etc.

Degree

Another important term:

Number of attributes in a relation.

Example:

STUDENT(ID, Name, Age, Dept)

Degree = 4

Cardinality of a relation

Number of tuples/rows.

If STUDENT contains 100 rows:

Cardinality = 100

🚩 Common Trap

Degree → Number of columns

Cardinality → Number of rows

9. Super Key

A super key is:

A set of one or more attributes that uniquely identifies each tuple in a relation.

Example:

STUDENT(Student_ID, Email, Name)

If both Student_ID and Email are unique:

Possible super keys include:

{Student_ID}

{Email}

{Student_ID, Name}

{Email, Name}

{Student_ID, Email}

{Student_ID, Email, Name}

All of these can uniquely identify a student.

But some contain unnecessary attributes.

10. Candidate Key

A candidate key is:

A minimal super key.

Minimal means:

No attribute can be removed while preserving uniqueness.

Suppose:

Student_ID → unique

Email → unique

Then:

{Student_ID}

{Email}

can both be candidate keys.

Important

A relation can have:

More than one candidate key.

11. Primary Key

A primary key is:

The candidate key selected to uniquely identify tuples.

If candidate keys are:

Student_ID

Email

we may select:

Student_ID

as the primary key.

The remaining candidate key can be called an:

Alternate key

Primary Key Properties

A primary key:

- uniquely identifies tuples
- cannot be NULL
- is based on a candidate key
- should be minimal
- can be composite

Composite Primary Key

A primary key may contain multiple attributes.

Example:

ENROLLMENT(Student_ID, Course_ID)

Primary key:

(Student_ID, Course_ID)

12. Foreign Key

A foreign key is an attribute/set of attributes in one relation that references a candidate/primary key of another relation (or potentially the same relation).

Example:

DEPARTMENT

Dept_ID Dept_Name

10 CS

20 IT

STUDENT

Student_ID Name Dept_ID

101 A 10

102 B 20

Here:

DEPARTMENT.Dept_ID

is the referenced key.

STUDENT.Dept_ID

is the foreign key.

Why Foreign Keys?

They represent relationships between tables and help enforce referential integrity.

13. Constraints

Constraints are rules that restrict invalid data.

Important SQL constraints:

13.1 NOT NULL

Prevents NULL values.

Name VARCHAR(50) NOT NULL

13.2 UNIQUE

Prevents duplicate values for the constrained column/set.

Email VARCHAR(100) UNIQUE

13.3 PRIMARY KEY

Provides unique identification.

A primary key is unique and cannot be NULL.

13.4 FOREIGN KEY

References a key in another relation.

13.5 CHECK

Restricts values according to a condition.

Age INT CHECK (Age >= 18)

13.6 DEFAULT

Provides a default value.

City VARCHAR(30) DEFAULT 'Pune'

14. Entity Integrity

Entity integrity states:

A primary key cannot contain NULL.

Why?

The primary key identifies an entity/tuple.

If:

Student_ID = NULL

the tuple cannot be uniquely identified using that key.

Remember

Entity Integrity

↓

Primary Key

↓

Cannot be NULL

15. Referential Integrity

Referential integrity ensures consistency between related tables.

Suppose:

DEPARTMENT

10 → CS

20 → IT

and:

STUDENT

101 → 10

102 → 20

103 → 50

Student 103 refers to Department 50.

But Department 50 doesn't exist.

Therefore:

Referential integrity is violated.

Important

A foreign key generally must:

- match an existing referenced key value, or
- be NULL when NULL is permitted by the schema.

Remember

Entity Integrity

→ Primary Key

→ NOT NULL

Referential Integrity

→ Foreign Key

→ Valid referenced value

16. ER Model

ER stands for:

Entity-Relationship

The ER model is a conceptual data model used to represent:

- entities
- attributes
- relationships
- constraints

It is often used during database design before implementing relational tables.

17. Entity

An entity is:

A distinguishable real-world object about which information is stored.

Examples:

- Student
 - Employee
 - Customer
 - Course
 - Department
 - Book
-

Entity Type vs Entity Instance

Entity Type

Defines the structure.

Example:

STUDENT

Entity Instance

A particular occurrence.

Example:

Student_ID = 101

Name = Harshali

18. Attributes

Attributes describe entities.

Example:

STUDENT

├— Student_ID

├— Name

├— DOB

├— Email

└— Phone

18.1 Simple Attribute

Cannot be meaningfully divided.

Example:

Age

18.2 Composite Attribute

Can be divided into smaller components.

Example:

Name

├— First_Name

├— Middle_Name

└— Last_Name

Address can also be composite.

18.3 Single-Valued Attribute

Has one value for an entity.

Example:

Date_of_Birth

18.4 Multivalued Attribute

Can have multiple values.

Example:

Phone_Number

One student may have multiple phone numbers.

18.5 Derived Attribute

Can be calculated from another attribute.

Example:

DOB → Age

Age is derived from Date of Birth.

18.6 Key Attribute

Uniquely identifies an entity.

Example:

Student_ID

19. Relationships

A relationship represents an association between entities.

Example:

STUDENT — ENROLLS — COURSE

Entities:

STUDENT

COURSE

Relationship:

ENROLLS

Another example:

EMPLOYEE — WORKS_FOR — DEPARTMENT

20. Degree of Relationship

Don't confuse this with **degree of a relation**.

Relationship degree means:

Number of participating entity types in a relationship.

Unary Relationship

One entity type participates.

Example:

EMPLOYEE — MANAGES — EMPLOYEE

An employee manages another employee.

Degree = 1

Binary Relationship

Two entity types participate.

STUDENT — ENROLLS — COURSE

Degree = 2

Most common.

Ternary Relationship

Three entity types participate.

SUPPLIER — SUPPLIES — PART

|

PROJECT

Degree = 3

21. Cardinality

Cardinality specifies:

How many instances of one entity can be associated with instances of another entity?

Main types:

21.1 One-to-One (1:1)

PERSON — PASSPORT

One person has one passport.

21.2 One-to-Many (1:N)

DEPARTMENT — STUDENT

One department can have many students.

21.3 Many-to-One (N:1)

Same relationship viewed from the opposite direction:

STUDENT — DEPARTMENT

Many students belong to one department.

21.4 Many-to-Many (M:N)

STUDENT — COURSE

One student can enroll in many courses.

One course can contain many students.

22. Participation Constraints

Participation specifies:

Whether participation of an entity in a relationship is mandatory or optional.

Two types:

22.1 Total Participation

Every entity instance must participate.

Example:

Every employee must belong to a department.

Therefore every Employee participates in:

WORKS_FOR

This is total participation of Employee.

22.2 Partial Participation

Participation is optional.

Example:

Not every employee manages a department.

Therefore some employees participate in:

MANAGES

and others don't.

This is partial participation.

23. Cardinality vs Participation

This distinction is **very important for application questions**.

Cardinality asks:

How many?

Examples:

1:1

1:N

M:N

Participation asks:

Must it participate?

Examples:

Total

Partial

They are independent concepts.

A relationship can simultaneously have:

1:N cardinality

+

Total participation on one side

+

Partial participation on the other side

24. Weak Entity

A weak entity is an entity that:

Cannot be uniquely identified by its own attributes alone.

It depends on a strong/owner entity for identification.

Example:

EMPLOYEE

|

| HAS

↓

DEPENDENT

Suppose:

EMPLOYEE

Employee_ID

Name

DEPENDENT

Dependent_Name

Age

Dependent_Name may not be unique across the entire company.

Therefore:

Employee_ID + Dependent_Name

can identify a dependent.

Characteristics of Weak Entity

A weak entity generally:

- doesn't have a complete key of its own
- depends on an owner entity
- has a partial key/discriminator
- participates totally in its identifying relationship

Strong vs Weak Entity

Strong Entity

Has its own key

Independent existence

Complete key

Can exist independently

Weak Entity

Cannot be uniquely identified independently

Depends on owner

Partial key/discriminator + owner key

Existence depends on owner

🚩 Exam Trap

A weak entity is **not simply an entity having a foreign key**.

The important property is:

It cannot be uniquely identified without its owner entity.

25. ER → Relational Mapping

This converts an ER model into relational tables.

This is one of the most important **application areas**.

25.1 Strong Entity → Relation

ER:

STUDENT

Student_ID

Name

Age

Relational model:

STUDENT(Student_ID, Name, Age)

The entity's key becomes the primary key.

25.2 Simple Attribute → Column

Example:

STUDENT

|— Student_ID

|— Name

└— Age

becomes:

STUDENT(Student_ID, Name, Age)

25.3 Composite Attribute → Components

ER:

Name

|— First_Name

└— Last_Name

Relation:

STUDENT(Student_ID, First_Name, Last_Name)

Usually the composite attribute itself is not stored as one atomic column.

25.4 Multivalued Attribute → Separate Relation

Suppose:

STUDENT

└— Phone_Number

and a student can have multiple phone numbers.

Create:

STUDENT(Student_ID, Name)

STUDENT_PHONE(Student_ID, Phone_Number)

A suitable primary key is commonly:

(Student_ID, Phone_Number)

25.5 1:1 Relationship

For a 1:1 relationship, the key of one entity can be included as a foreign key in the other relation.

Example:

PERSON 1 — 1 PASSPORT

Possible representation:

PERSON(Person_ID, Name, Passport_ID)

where:

Passport_ID → Foreign Key

Which side should contain the FK can depend on participation and design considerations.

25.6 1:N Relationship

This is a **must-remember rule**:

Place the primary key of the 1-side into the relation representing the N-side as a foreign key.

Example:

DEPARTMENT 1 — N STUDENT

Relations:

DEPARTMENT(Dept_ID, Dept_Name)

STUDENT(Student_ID, Name, Dept_ID)

Here:

STUDENT.Dept_ID

is the foreign key.

Memory:

1:N → FK goes to N side

25.7 M:N Relationship

For a many-to-many relationship:

Create a separate relation.

Example:

STUDENT M — N COURSE

Create:

STUDENT(Student_ID, Name)

COURSE(Course_ID, Course_Name)

ENROLLMENT(Student_ID, Course_ID)

Usually:

PRIMARY KEY(Student_ID, Course_ID)

Both are foreign keys.

Memory:

M:N → New table

25.8 Weak Entity Mapping

Suppose:

EMPLOYEE

|

↓

DEPENDENT

Create:

EMPLOYEE(Employee_ID, Name)

DEPENDENT(Employee_ID, Dependent_Name, Age)

A common key for the weak entity relation is:

(Employee_ID, Dependent_Name)

where:

Employee_ID

is also a foreign key referencing EMPLOYEE.

★ **Final High-Yield Revision Sheet**

Before your exam, remember these relationships:

Three Schema

External → User view

Conceptual → Logical structure

Internal → Physical storage

Data Independence

Physical → Internal changes

Logical → Conceptual changes

Relational Model

Relation → Table

Tuple → Row

Attribute → Column

Domain → Allowed values

Degree → Number of columns

Cardinality → Number of rows

Keys

Super Key → Unique

Candidate Key → Minimal Super Key

Primary Key → Selected Candidate Key

Foreign Key → References another relation's key

Integrity

Entity → Primary Key → NOT NULL

Referential → Foreign Key → Valid reference

ER

Entity → Object

Attribute → Property

Relationship → Association

Cardinality → How many?

Participation → Mandatory/optional

ER Mapping

Strong Entity → Table

Composite → Components

Multivalued → Separate table

1:N → FK on N side

M:N → New table

Weak Entity → Owner key + partial key